QVANN: A Quantization-Driven Authenticated Index for Efficient Verifiable ANN Search

Dong Wang
dong.wang@chd.edu.cn
Chang'an University

Ningning Cui
willber1988@163.com
Chang'an University

Xiangyu Song
xiangyu.song@chd.edu.cn
Chang'an University

Jianxin Li
jianxin.li@ecu.edu.au
Edith Cowan University

Jianzhong Qi
jianzhong.qi@unimelb.edu.au
The University of Melbourne

ABSTRACT

Outsourced vector databases have become increasingly important for large-scale similarity search, enabling efficient retrieval without local data management. However, outsourcing query execution to untrusted servers raises critical concerns about result correctness and completeness, motivating verifiable approximate nearest neighbor (ANN) search. Existing authenticated ANN methods incur substantial communication overhead due to large verification objects (VOs). We identify two major sources of this overhead: excessive high-dimensional vector disclosure and fragmented Merkle authentication. To address these challenges, we propose QVANN, a quantization-driven authenticated index that co-designs ANN search and authentication for efficient verification. Unlike conventional approaches, QVANN decouples coarse-precision graph traversal from progressive result certification, substantially reducing vector disclosure while preserving high recall. It further employs expansion-centric authentication to minimize Merkle proof overhead. We establish its correctness and completeness and evaluate it on four real-world datasets. Experimental results show that QVANN outperforms state-of-the-art methods with up to 90.7% smaller VOs and 4.2× faster verification at comparable recall.

PVLDB Reference Format:
Dong Wang, Ningning Cui, Xiangyu Song, Jianxin Li, and Jianzhong Qi. QVANN: A Quantization-Driven Authenticated Index for Efficient Verifiable ANN Search. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 INTRODUCTION

The rapid growth of high-dimensional vector data has made approximate nearest neighbor (ANN) search fundamental to applications such as information retrieval, recommendation, and retrieval-augmented generation [24, 30, 31]. To meet the demand for scalable vector search, managed vector database services, such as Zilliz Cloud [46], Weaviate Cloud [38], and Qdrant Cloud [28], allow data owners to outsource vector storage, index maintenance, and query processing to third-party servers. However, outsourcing places query execution beyond the direct control of data owners and users, raising concerns about result integrity. An untrusted server may skip costly search operations, omit valid results, or deliberately manipulate query results for its own benefit [20, 30].

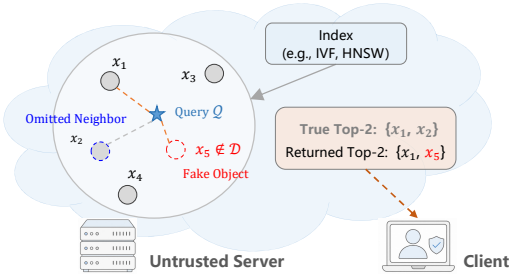


Figure 1: Motivation of verifiable ANN search. An untrusted server may return a forged object $\mathbf{x}_5 \notin \mathcal{D}$ while omitting $\mathbf{x}_2$ from the expected top-2 results.

Example 1. *Fig. 1 illustrates such a case. Suppose an honest execution of the prescribed top-2 ANN query returns $\{\mathbf{x}_1, \mathbf{x}_2\}$. (i) To manipulate the query outcome for its own benefit, the server may return a fabricated object $\mathbf{x}_5 \notin \mathcal{D}$, violating correctness. (ii) To reduce computation, the server may skip required search steps and omit $\mathbf{x}_2$, violating completeness.*

To detect these violations, the query user must verify (i) *correctness*, ensuring that returned objects are authentic and satisfy the prescribed query protocol, and (ii) *completeness*, ensuring that no result required by an honest execution of that protocol is omitted. This motivates *verifiable ANN search*, which aims to ensure query integrity without requiring users to rerun the outsourced search.

Existing approaches to verifiable ANN search mainly fall into two categories. (i) *Zero-knowledge (ZK) proof-based methods*, such as V3DB and Atlas [2, 30], prove ANN query execution over committed indexes. Despite producing succinct proofs, they incur substantial proof-generation costs. For example, V3DB requires 29.24 s to generate a proof on SIFT1M with 8,192 inverted lists. (ii) *Authenticated data structure (ADS)-based methods.* ADS-based query authentication has a longer history in outsourced databases, supporting spatial top-k and exact kNN queries [8, 41], Boolean range queries [39], and image retrieval [13]. However, these methods rely on low-dimensional spatial bounds, query-specific predicates, or image representations, making them difficult to apply directly to

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Table 1: VO breakdown comparison of QVANN and existing ADS-based methods (VO unit: KB).

Method	SIFT		NYTimes		GloVe		DEEP	
	VO_{trav}	VO_{auth}	VO_{trav}	VO_{auth}	VO_{trav}	VO_{auth}	VO_{trav}	VO_{auth}
MR-tree	235,091	123	282,959	0.3	466,929	3.7	373,908	15.3
ANNProof	1,196	689.3	11,670	1,996	30,366	10,208	1,367	942.5
VLAH	11,636	2,315	131,355	4,134	141,334	14,751	32,253	6,069
IVF-ADS	20,146	4,546	168,056	3,100	120,488	14,206	14,592	4,626
NSW-ADS	742.1	225.9	14,164	1,532	21,906	4,432	958.5	351.6
Ours	108.5	93.7	870.9	400.3	2,086	1,746	113.2	157.9

Notes. Gray shading and underlining indicate the lowest and second-lowest costs, respectively. MR-tree performs exact kNN search with 100% recall, while the other methods are tuned to achieve approximately 95% Recall@10. Dataset details, baseline configurations, and default parameter settings are provided in Sec. 7.

generic high-dimensional ANN search. More recent methods, such as ANNProof and VLAH [20, 36], extend ADS-based authentication to high-dimensional ANN search using Merkle proofs. Although avoiding expensive ZK proving, they incur substantial communication overhead. For example, ANNProof generates a VO of approximately 2.8 MB on DEEP1M with result size k=50, requiring about 0.22 s for transmission alone over a 100 Mbps network [36].

The large VO mainly arises from two sources. (i) *High-dimensional vector disclosure.* In conventional NSW, verifying each expansion requires reproducing distance-based candidate ordering, while IVF must certify the exclusion of candidates in probed lists. Both require disclosing numerous vectors beyond the final top-k results. Consequently, the traversal payload VO_{trav} grows with the number of accessed candidates and their dimensionality. (ii) *Fragmented authentication.* Conventional Merkle authentication treats graph records as separate proof targets, requiring numerous hashes to authenticate scattered accesses and increasing VO_{auth}.

Table 1 quantifies these overheads across four datasets. Even the best existing ANN-based ADS methods incur up to 21.9 MB of traversal payload and 4.4 MB of authentication proofs. Moreover, MR-tree, an authenticated exact kNN index, incurs over 15× the traversal payload of ANNProof across all datasets. These results highlight the need to reduce both vector disclosure and authentication overhead. To address these limitations, we propose QVANN, a quantization-driven authenticated index for efficient verifiable ANN search. We identify three main challenges:

Challenge 1: How can quantization reduce vector disclosure while maintaining high recall? Quantization is widely used to compress high-dimensional vectors, but introduces distance errors that may degrade ANN recall [10, 15]. More critically, these errors may cause incorrect candidate exclusion, undermining result verification. To address this, we propose a *quantization-driven verification framework* that progressively refines vector representations. By establishing safe distance bounds at different precision levels, it excludes most non-result candidates using coarse representations while refining only ambiguous ones. This substantially reduces vector disclosure while ensuring correct top-k selection among searched candidates. **Challenge 2: How can graph traversal fully exploit quantization?** Although progressive quantization enables efficient candidate verification, directly applying it to NSW offers limited benefits. Unlike IVF, where candidates are verified after index probing, verifying NSW traversal requires repeated vector refinement to reproduce exact-distance decisions, largely

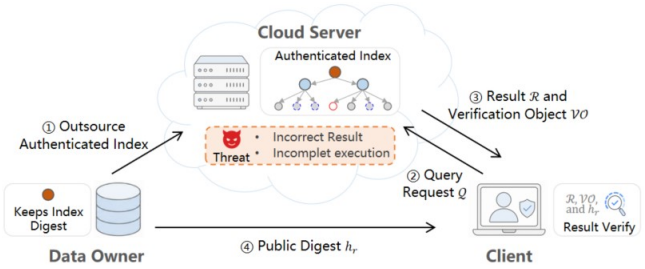


Figure 2: System model.

offsetting the benefits of quantization. To address this, we combine the strengths of IVF and NSW by introducing *verification-decoupled traversal*, which drives graph exploration using coarse-precision representations and reserves exact distances for final result determination. This decoupling allows the verifier to replay traversal without fine-grained refinement while progressively certifying the final candidate set. Consequently, QVANN combines efficient graph exploration with compact verification evidence, substantially reducing VO_{trav}. **Challenge 3: How can graph accesses be authenticated efficiently?** Conventional record-wise authentication produces fragmented Merkle proofs that poorly match graph traversal. We design an *expansion-centric authenticated index* that jointly authenticates routing information and separates refinement data for on-demand disclosure. Graph-aware record ordering further improves proof sharing, reducing VO_{auth}.

In summary, our main contributions are as follows:

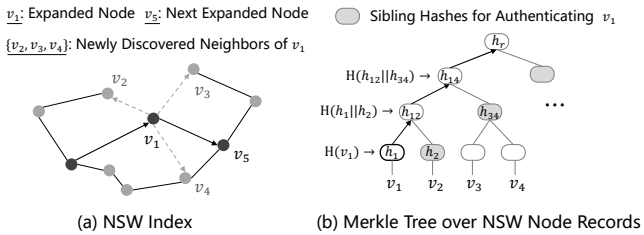
- We propose QVANN, a verification-oriented ANN index. To the best of our knowledge, it is the first to jointly redesign quantized graph traversal and authentication for verifiable ANN search.
- We develop a novel quantization-driven verification framework that substantially reduces VO size while preserving high recall.
- We design a VO-efficient authenticated index tailored to verifiable ANN search, reducing Merkle proof overhead through expansion-centric authentication.
- We conduct theoretical analysis and extensive experiments, demonstrating that QVANN outperforms state-of-the-art methods with up to 90.7% smaller VOs and 4.2× faster verification.

2 PROBLEM FORMULATION

System Model. As illustrated in Fig. 2, our system comprises three entities: (i) data owner (DO), (ii) cloud server (CS), and (iii) query user (QU). Their responsibilities are as follows.

- *Data Owner.* Given a vector dataset $\mathcal{D}$, DO is responsible for constructing an authenticated ANN index $\mathcal{I}$, outsourcing $\mathcal{D}$ and $\mathcal{I}$ to CS, and maintaining a trusted digest Digest (Step 1).
- *Cloud Server.* CS is responsible for storing the outsourced data and executing the prescribed ANN query protocol. Given a query Q, CS processes the query over index $\mathcal{I}$ and returns a result set $\mathcal{R}$ together with a verification object $\mathcal{VO}$ (Step 3).
- *Query User.* QU submits Q to CS (Step 2) and obtains Digest from DO through an authenticated channel (Step 4). QU then verifies $\mathcal{R}$ using $\mathcal{VO}$ and Digest.

Threat Model. Following prior work [8, 20, 36], we assume that DO and QU are fully trusted, while CS is untrusted and may forge



(a) NSW Index (b) Merkle Tree over NSW Node Records

Figure 3: A verifiable ANN baseline combining (a) an NSW graph with (b) Merkle authentication.

or tamper with results, omit valid objects, or deviate from the prescribed ANN query protocol. To ensure query integrity, QU verifies two properties:

- *Correctness.* All returned results must originate from the authenticated dataset and remain unmodified.
- *Completeness.* All results required by an honest execution of the prescribed ANN query protocol must be returned without omission.

The detailed security notions will be formalized when we perform security analysis in Section V-A

Problem definition. Based on the above system and threat models, we study how to support efficient verifiable ANN search over an untrusted cloud server while minimizing the communication overhead. We formally define as follows:

Definition 1 (Verifiable ANN Query). Let $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$ be a dataset of n d-dimensional vectors, and let $\mathcal{I}$ denote an authenticated ANN index constructed over $\mathcal{D}$. Given a query $Q = (\mathbf{q}, k)$, where $\mathbf{q} \in \mathbb{R}^d$ is the query vector and k is the requested result size, let C denote the candidate set reached by the prescribed ANN query protocol over $\mathcal{I}$. CS returns a result set $\mathcal{R} = \{r_1, \ldots, r_k\}$ together with a verification object $\mathcal{VO}$. Based on $\mathcal{VO}$, it enables QU to guarantee (i) *Correctness.* $\forall r \in \mathcal{R},\ r \in \mathcal{D} \wedge \mathrm{rank}_C(r) \leq k$; (ii) *Completeness.* $\forall x \notin \mathcal{R} \wedge x \in C,\ \mathrm{rank}_C(x) > k$, where $\mathrm{rank}_C(x)$ denotes the rank of x in C according to its distance to $\mathbf{q}$.

3 PRELIMINARIES

3.1 NSW Index

Navigable Small World (NSW) is a classical graph-based ANN index. Given a vector dataset $\mathcal{D}$, NSW organizes the data objects as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where each vertex $v_i \in \mathcal{V}$ corresponds to a vector $\mathbf{x}_i \in \mathcal{D}$, and each edge $(v_i, v_j) \in \mathcal{E}$ indicates that v_j is a neighbor of v_i. Given a query vector $\mathbf{q}$, NSW starts from an entry node and iteratively explores the graph. The discovered nodes are maintained in a priority queue according to their distances to $\mathbf{q}$. At each step, the closest unexpanded node is popped from the queue and expanded, and its newly discovered neighbors are inserted into the queue. In Fig. 3(a), v_1 is the currently expanded node, and its newly discovered neighbors $\{v_2, v_3, v_4\}$ are inserted into the candidate queue. Among the candidate nodes in the queue, v_5 is then selected as the next node to expand. This process continues until the prescribed search condition is satisfied, e.g., after expanding *efSearch* nodes. Finally, the top-k nearest candidates are returned as the ANN results.

3.2 Merkle Tree

Merkle tree is a widely used ADS for verifying the integrity of outsourced data. Its root hash h_r commits to all records stored in the tree. As shown in Fig. 3(b), we construct a Merkle tree over the NSW index, where each leaf corresponds to an authenticated NSW record, such as a node v_i or an edge (v_i, v_j). Consider the authentication of v_1. The server returns v_1 together with the sibling hashes along its authentication path. The client first computes $h'_1 = H(v_1)$ and then reconstructs the root hash as $h'_{12} = H(h'_1 \| h_2)$ and $h'_r = H(h'_{12} \| h_{34})$, where h_2 and h_{34} are the proof hashes returned by the server. The record is accepted only if $h'_r = h_r$. Therefore, even a small modification to v_1 changes the reconstructed root hash and can be detected by the client.

3.3 Baseline

ADS-based verifiable ANN approaches typically combine conventional ANN indexes with authenticated data structures [20, 36]. For example, the NSW index in Fig. 3(a) can be authenticated using the Merkle tree in Fig. 3(b), forming a straightforward verifiable NSW baseline. The cloud executes conventional NSW search, while Merkle proofs authenticate the accessed index records [20].

To verify the search execution and returned results, the user needs to replay the NSW search process. Therefore, the cloud returns the VO consisting of two parts:

- VO_{trav}: The accessed graph records, including nodes, vectors, and edges, enabling QU to reproduce the traversal and verify search execution.
- VO_{auth}: The Merkle proofs authenticating the disclosed records. Multi-proofs reduce proof size by sharing common authentication paths.

However, this baseline incurs substantial VO overhead due to two bottlenecks. (i) *Vector disclosure.* Replaying NSW traversal requires the high-dimensional vectors involved in distance-based expansion decisions, resulting in a large VO_{trav}. (ii) *Authentication overhead.* Accessed graph records are scattered across Merkle leaves, requiring numerous proof hashes and increasing VO_{auth}. Similar overhead can arise in straightforward Merkle-authenticated IVF and hybrid IVF–NSW designs. Table 1 illustrates these two sources of overhead across representative methods.

4 FRAMEWORK

To address the two bottlenecks of existing baselines, namely (i) excessive vector disclosure and (ii) high authentication overhead, we next present the framework of QVANN, which tackles these bottlenecks through three complementary designs.

4.1 Progressive Quantization Verification

The baseline requires disclosing numerous raw vectors in VO_{trav} to replay graph traversal. We therefore introduce a quantization-driven verification framework. Quantization is widely used to compress vectors for efficient ANN search, typically at the cost of reduced recall [10, 15]. In contrast, we exploit quantization to reduce vector disclosure for verification while maintaining high recall.

Definition 2 (Quantized Representation). Given a vector $\mathbf{x} \in \mathbb{R}^d$, b-bit quantization maps each coordinate x_j to a quantization code.

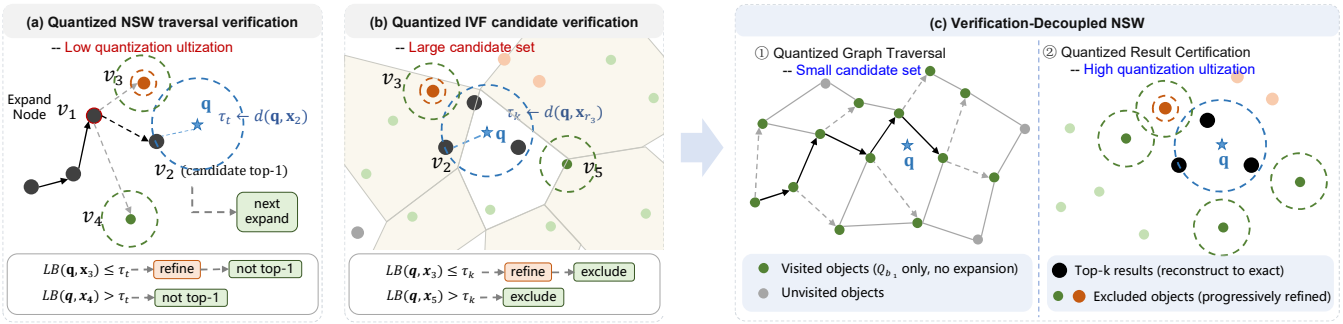


Figure 4: Comparison of progressive quantization for ANN verification. (a) Quantized NSW accesses few candidates but requires repeated refinement during traversal. (b) Quantized IVF enables efficient refinement but accesses many candidates. (c) Our verification-decoupled NSW combines both advantages through coarse-precision traversal and progressive result certification.

Each b-bit code corresponds to a quantization interval $[l_j^{(b)}, u_j^{(b)}]$ containing x_j. Accordingly, we denote the b-bit quantized representation of $\mathbf{x}$ as $Q_b(\mathbf{x}) = \prod_{j=1}^{d}[l_j^{(b)}, u_j^{(b)}]$. A larger b provides a finer representation with a smaller quantization region.

Progressive quantization. Rather than using a single quantization precision, we configure a sequence of quantization levels $\mathcal{B} = \{b_1, b_2, \ldots, b_L\}$, where $b_1 < b_2 < \cdots < b_L$. The quantized representations are constructed to be progressively nested as $Q_{b_1}(\mathbf{x}) \supseteq Q_{b_2}(\mathbf{x}) \supseteq \cdots \supseteq Q_{b_L}(\mathbf{x}) \ni \mathbf{x}$. Thus, verification can start from the coarsest representation $Q_{b_1}(\mathbf{x})$ and disclose a finer representation only when the current precision is insufficient.

Based on these quantization regions, we obtain the following property for candidate verification.

Lemma 1 (Quantized Candidate Exclusion)**.** *Given a query vector* $\mathbf{q}$ *and a quantized representation* $Q_b(\mathbf{x})$*, define the distance lower bound as* $LB_b(\mathbf{q}, \mathbf{x}) = \min_{\mathbf{z} \in Q_b(\mathbf{x})} d(\mathbf{q}, \mathbf{z})$*. Since* $\mathbf{x} \in Q_b(\mathbf{x})$*, we have* $LB_b(\mathbf{q}, \mathbf{x}) \le d(\mathbf{q}, \mathbf{x})$*. Hence, for any threshold* τ*,* $LB_b(\mathbf{q}, \mathbf{x}) > \tau$ *implies* $d(\mathbf{q}, \mathbf{x}) > \tau$*. Moreover, for nested quantization levels* $Q_{b_i}(\mathbf{x}) \supseteq Q_{b_{i+1}}(\mathbf{x})$*, we have* $LB_{b_i}(\mathbf{q}, \mathbf{x}) \le LB_{b_{i+1}}(\mathbf{q}, \mathbf{x}) \le d(\mathbf{q}, \mathbf{x})$*.*

Lemma 1 enables candidate verification using coarse quantized representations and refines only when necessary. We next apply it to NSW and IVF.

Quantized NSW traversal verification. Fig. 4(a) illustrates its application to NSW traversal replay. Suppose v_1 is the currently expanded node and $\mathcal{P} = \{v_2, v_3, v_4\}$ is the candidate queue, where v_2 is the current top-1 candidate. Let $\tau_t = d(\mathbf{q}, \mathbf{x}_2)$ denote its exact distance. To verify that v_2 should be expanded next, each competing candidate $u \in \mathcal{P} \setminus \{v_2\}$ only needs to prove $d(\mathbf{q}, \mathbf{x}_u) > \tau_t$. By Lemma 1, this condition is certified whenever $LB_b(\mathbf{q}, \mathbf{x}_u) > \tau_t$. For example, v_4 is directly certified not to precede v_2 because $LB_b(\mathbf{q}, \mathbf{x}_4) > \tau_t$. For v_3, the current representation gives $LB_b(\mathbf{q}, \mathbf{x}_3) \le \tau_t$, and thus a finer representation is disclosed until its ordering relative to v_2 can be determined. Hence, progressive quantization allows competing candidates to be verified using coarse representations whenever possible, avoiding unnecessary disclosure of raw vectors.

Quantized IVF candidate verification. Fig. 4(b) illustrates its application to IVF candidate verification. After the probed lists generate the candidate set C, the top-k results $\mathcal{R}$ determine the exact threshold $\tau_k = d(\mathbf{q}, \mathbf{x}_{r_k})$. For each non-result candidate $v \in C \setminus \mathcal{R}$, it suffices to verify $d(\mathbf{q}, \mathbf{x}_v) > \tau_k$. By Lemma 1, v can be safely excluded once $LB_b(\mathbf{q}, \mathbf{x}_v) > \tau_k$. For example, v_6 is directly excluded at the current precision, whereas v_5 requires a finer representation because $LB_b(\mathbf{q}, \mathbf{x}_5) \le \tau_k$. Hence, progressive quantization allows non-result candidates to be certified at coarse precision whenever their lower bounds already exceed the final top-k threshold.

Analysis. Let $r_b(v, \tau) = \mathbb{I}[LB_b(\mathbf{q}, v) \le \tau]$ indicate whether the current b-bit representation of v is insufficient for exclusion and thus requires refinement. (i) For NSW, this test is repeatedly performed during traversal, giving $\rho_{\text{NSW}} = \frac{\sum_{t=1}^{E} \sum_{v \in \mathcal{P}_t \setminus \{v_t\}} r_b(v, \tau_t)}{\sum_{t=1}^{E} (|\mathcal{P}_t| - 1)}$, where v_t and τ_t denote the top-1 candidate and its exact distance at the t-th expansion. (ii) For IVF, each non-result candidate is checked once against the final top-k threshold, yielding $\rho_{\text{IVF}} = \frac{\sum_{v \in C \setminus \mathcal{R}} r_b(v, \tau_k)}{|C \setminus \mathcal{R}|}$.

The two formulations reveal distinct refinement behaviors: (i) NSW repeatedly verifies local top-1 decisions during traversal, whereas (ii) IVF verifies candidates once against the final threshold τ_k. Detailed in Sec. 7.7, quantization reduces VO size by up to 79.7% for IVF, compared with only 33.4% for NSW. Nevertheless, quantized IVF still produces up to 11.2× larger VOs than quantized NSW due to its substantially larger candidate set.

4.2 Verification-Decoupled Traversal

Motivated by the above analysis, we combine the complementary advantages of NSW and IVF: the small candidate set of NSW and the high quantization utilization of IVF. To this end, we redesign the NSW protocol to be verification-friendly with two steps: *graph traversal* and *result certification*, as illustrated in Fig. 4(c).

Quantized Graph Traversal. Instead of reproducing an NSW traversal defined by exact distances, we directly define traversal using the coarsest quantization level b_1. Given $Q_{b_1}(\mathbf{x}_v)$, let $\hat{\mathbf{x}}_v^{b_1}$ denote its reconstructed midpoint vector. We define the routing score as $\kappa_{\mathbf{q}}(v) = d(\mathbf{q}, \hat{\mathbf{x}}_v^{b_1})$. Discovered nodes are ordered by $(\kappa_{\mathbf{q}}(v), id_v)$, and the closest unexpanded node is selected until E nodes have been expanded or the candidate queue becomes empty. Since all routing decisions depend solely on coarse representations, no higher-precision refinement is needed during traversal. The resulting candidate set C contains only nodes discovered through these expansions.

Theorem 1 (Error-Bounded Coverage of Coarse Routing). *Consider exact and coarse routing over the same graph and entry node, using best-first expansion without candidate pruning. Let $\mathcal{X}_e$ and C_e denote the expanded and discovered sets of exact routing with budget E_e. Suppose the reconstruction error satisfies $\|\mathbf{x}_v - \hat{\mathbf{x}}_v^{b_1}\|_2 \le \epsilon$ for all v. Define $D = \max_{v\in\mathcal{X}_e} \|\mathbf{q} - \mathbf{x}_v\|_2, T = \max_{v\in\mathcal{X}_e} \kappa_{\mathbf{q}}(v)$. Let $\mathcal{S}_T$ be the entry-reachable set in the subgraph induced by $\{v : \kappa_{\mathbf{q}}(v) \le T\}$. Then, $|\mathcal{S}_T| \le |\{v : \|\mathbf{q}-\mathbf{x}_v\|_2 \le D+2\epsilon\}|$. If coarse routing uses an expansion budget $E_c \ge |\mathcal{S}_T|$, its discovered set C_c satisfies $C_e \subseteq C_c$. Consequently, exact top-k selection over these sets yields $R_c@k \ge R_e@k$.*

Proof. By the triangle inequality, $|\kappa_{\mathbf{q}}(v) - \|\mathbf{q} - \mathbf{x}_v\|_2| \le \epsilon$. Hence, $T \le D + \epsilon$, and every $v \in \mathcal{S}_T$ satisfies $\|\mathbf{q} - \mathbf{x}_v\|_2 \le T + \epsilon \le D+2\epsilon$, establishing the bound on $|\mathcal{S}_T|$. Every node in $\mathcal{X}_e$ is reachable from the entry through previously expanded nodes, all having routing scores at most T. Thus, $\mathcal{X}_e \subseteq \mathcal{S}_T$. Suppose coarse routing expands a node w outside $\mathcal{S}_T$ before exhausting $\mathcal{S}_T$. Consider the first such expansion. Since all preceding expansions belong to $\mathcal{S}_T$, node w must have been discovered from $\mathcal{S}_T$. Hence, $\kappa_{\mathbf{q}}(w) > T$; otherwise, w would also belong to $\mathcal{S}_T$. Meanwhile, an unexpanded node in $\mathcal{S}_T$ implies the existence of a discovered frontier node $u \in \mathcal{S}_T$ with $\kappa_{\mathbf{q}}(u) \le T$. This contradicts the best-first expansion rule. Therefore, coarse routing expands all nodes in $\mathcal{S}_T$ within $|\mathcal{S}_T|$ expansions, including $\mathcal{X}_e$. Since both protocols discover all neighbors of expanded nodes, $C_e \subseteq C_c$. Exact top-k selection over a superset cannot decrease recall. □

Theorem 1 establishes that coarse routing preserves candidate coverage under a sufficient expansion budget. Experiments in Sec. 7.9 further show that coarse routing achieves comparable or higher recall than exact routing even at the same expansion budget.

Quantized Result Certification. After traversal, the server computes exact distances for all candidates in C to obtain the top-k results $\mathcal{R}$ and the threshold $\tau = d(\mathbf{q}, \mathbf{x}_{r_k})$. Unlike traversal verification, result certification compares each non-result candidate against this fixed threshold. Specifically, for each $v \in C \setminus \mathcal{R}$, verification starts from $Q_{b_1}(\mathbf{x}_v)$. If $LB_{b_i}(\mathbf{q}, v) > \tau$, the candidate is safely excluded; otherwise, its representation is progressively refined until exclusion is certified or the exact distance is recovered. In the latter case, exclusion is determined by the exact distance and node-ID ordering. Thus, higher-precision information is disclosed only for candidates that cannot be excluded at coarser levels.

Analysis. The decoupled protocol combines the candidate efficiency of NSW with the quantization efficiency of IVF. First, fixing the routing precision at b_1 eliminates higher-precision refinement during traversal, yielding $\rho_{\text{trav}} = 0$. Second, result verification is deferred until the candidate set C is formed, so progressive quantization is applied only to $|C \setminus \mathcal{R}|$ non-result candidates against the fixed threshold τ. Consequently, our design retains the small candidate set of NSW while achieving IVF-like quantization utilization during result certification.

4.3 Expansion-Centric Authentication

Although verification-decoupled traversal reduces vector disclosure, authenticating scattered routing records remains costly. We observe a mismatch between graph execution and conventional authentication: graph search processes data by node expansion, whereas record-wise authentication treats the accessed routing information as independent Merkle leaves. We address this mismatch by aligning authentication granularity with graph execution and separating routing from refinement authentication.

Expansion-Centric Routing Authentication. Expanding a node v requires the identifiers and coarse representations of its neighbors. Instead of authenticating these records separately, we consolidate them into an expansion record $\mathsf{ER}(v)$, authenticated as a single Merkle leaf. Thus, one authenticated expansion record provides the routing information required to reproduce the corresponding graph expansion.

Let $\mathcal{A}(v)$ denote the record-wise authentication leaves accessed when expanding v. For an expansion set $\mathcal{X}$, the number of routing proof targets changes from $T_{\text{rec}} = |\bigcup_{v\in\mathcal{X}} \mathcal{A}(v)|$ to $T_{\text{exp}} = |\mathcal{X}|$. This makes the number of routing authentication targets depend on graph expansions rather than individual neighbor-record accesses.

Precision-Separated Authentication. Routing and result certification access vector representations at different precision levels. We therefore authenticate coarse routing information and higher-precision refinements using separate Merkle trees. Let $\mathcal{F}_i$ denote the candidates requiring authentication at refinement level b_i. The total number of refinement proof targets is $\sum_i |\mathcal{F}_i|$. Candidates excluded at coarser levels require no authentication at finer levels. Refinement records are arranged in BFS order to promote Merkle path sharing among graph-local accesses.

Analysis. Expansion-centric authentication reduces the number of routing proof targets in our evaluated workloads, while precision separation avoids authenticating unused refinement records. In our preliminary experiments, the number of routing targets decreases from 880.11 to 68.25, reducing Merkle proof size from 198.80 to 58.36 KiB. These results demonstrate the benefit of aligning authentication granularity with the information required by graph execution.

5 OUR QVANN

In this section, we present the detailed design of QVANN, including its index structure, verifiable query processing, and verification procedure.

5.1 Index Structure

As shown in Fig. 5, the QVANN index consists of two components: (i) a quantized graph index for ANN search and VO_{trav} generation, and (ii) Merkle Trees with expansion-centric authentication for VO_{auth} generation.

Quantized Graph Index. The graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ follows the NSW structure introduced in Sec. 3.1. To support progressive quantization and verification-decoupled traversal, each node maintains a coarse routing representation together with multi-level refinement information.

Definition 3 (QVANN Node). Given the configured quantization levels $\mathcal{B} = \{b_1, b_2, \ldots, b_L\}$, where $b_1 < b_2 < \cdots < b_L$, each node $v \in \mathcal{V}$ contains:

- id_v: the identifier of node v;
- $\mathbf{z}_v^{b_1}$: the coarsest b_1-bit quantization code corresponding to $Q_{b_1}(\mathbf{x}_v)$, which is used for graph traversal;

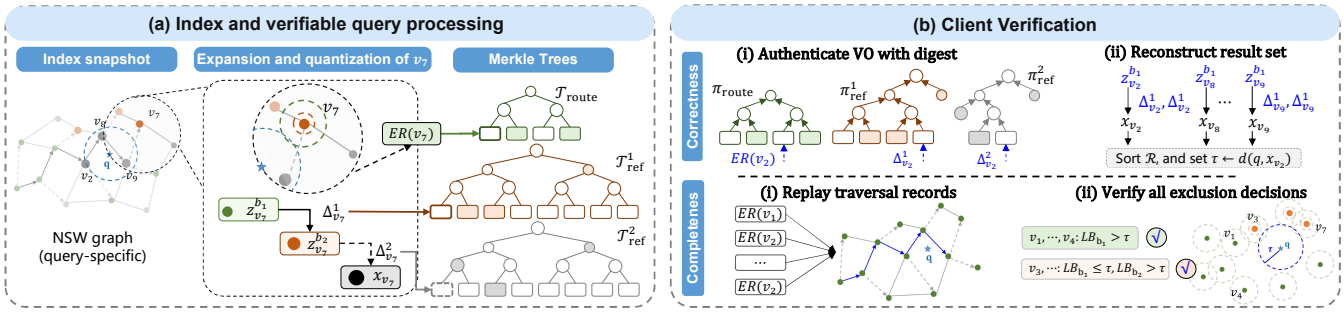


Figure 5: A running example of QVANN. (a) Quantized graph search and VO construction using expansion records and progressive refinements authenticated by separate Merkle trees. (b) Client-side verification through VO authentication, result reconstruction, traversal replay, and progressive candidate exclusion.

- Δ_v^i: the refinement information at level i. For $1 \le i < L$, Δ_v^i refines $\mathbf{z}_v^{b_i}$ to $\mathbf{z}_v^{b_{i+1}}$. For $i = L$, Δ_v^L refines $\mathbf{z}_v^{b_L}$ to the exact vector $\mathbf{x}_v$.

The coarsest code $\mathbf{z}_v^{b_1}$ is sufficient for routing, whereas $\{\Delta_v^i\}_{i=1}^L$ is disclosed progressively for result certification.

Expansion-Centric Authentication Index. Following Sec. 4.3, QVANN authenticates graph accesses at the granularity of one expansion rather than individual node and edge records.

Definition 4 (Expansion Record). For each node $v \in \mathcal{V}$, its expansion record is $\mathsf{ER}(v) = \langle id_v, [(id_u, \mathbf{z}_u^{b_1})]_{u \in \mathcal{N}(v)} \rangle$, where $\mathcal{N}(v)$ denotes the neighbor set of v. The neighbors follow a canonical order, e.g., ascending node ID.

The expansion record contains exactly the routing information required when v is expanded.

Definition 5 (QVANN Authentication Index). The authentication index $\mathcal{I}_{\text{auth}}$ consists of authenticated entry metadata, one routing Merkle tree, and L refinement Merkle trees:

- $\mathsf{M}_{\text{entry}}$: the entry node identifier id_s and its coarsest quantization code $\mathbf{z}_s^{b_1}$;
- $\mathcal{T}_{\text{route}}$: the routing authentication tree over $\{\mathsf{ER}(v) \mid v \in \mathcal{V}\}$;
- $\mathcal{T}^i_{\text{ref}}$: the refinement authentication tree over $\{\Delta_v^i \mid v \in \mathcal{V}\}$, for $1 \le i \le L$.

Each authentication tree is a Merkle tree. For the routing tree, the leaf hash of node v is $h_v^{\text{route}} = H(\mathsf{ER}(v))$. For the i-th refinement tree, the corresponding leaf hash is $h_v^{\text{ref},i} = H(id_v \| \Delta_v^i)$. Each non-leaf node with child hashes h_l and h_r stores $H(h_l \| h_r)$.

The routing tree $\mathcal{T}_{\text{route}}$ arranges expansion records by node ID, while refinement trees $\mathcal{T}^i_{\text{ref}}$ use a deterministic BFS ordering to improve authentication locality. Let r_{route} and $\{r^i_{\text{ref}}\}_{i=1}^L$ denote the corresponding Merkle roots. The DO publishes the trusted digest $\text{Digest} = \langle \mathsf{M}_{\text{entry}}, \mathsf{M}_{\text{quant}}, r_{\text{route}}, \{r^i_{\text{ref}}\}_{i=1}^L \rangle$, where $\mathsf{M}_{\text{quant}}$ specifies the quantization parameters and decoding rules.

Example 2. *Fig. 5(a) shows the QVANN index, where node v_7 is selected to illustrate its stored information. (i) In the quantized graph, v_7 maintains the coarse routing $\mathbf{z}_{v_7}^{b_1}$ together with its progressive refinement information $\Delta_{v_7}^1$ and $\Delta_{v_7}^2$. (ii) In the Merkle trees, the expansion record of v_7, i.e., $\mathsf{ER}(v_7) = \langle id_{v_7}, [(id_{v_9}, \mathbf{z}_{v_9}^{b_1}), (id_{v_{10}}, \mathbf{z}_{v_{10}}^{b_1}), (id_{v_{11}}, \mathbf{z}_{v_{11}}^{b_1})] \rangle$, is authenticated as a leaf node in $\mathcal{T}_{\text{route}}$. Meanwhile, $\Delta_{v_7}^1$ and $\Delta_{v_7}^2$ are stored as leaf nodes in $\mathcal{T}^1_{\text{ref}}$ and $\mathcal{T}^2_{\text{ref}}$, respectively.*

Algorithm 1: QVANN.Query$(\mathbf{q}, k, E)$

Input: Query vector $\mathbf{q}$, result size k, expansion budget E
Output: Result $\mathcal{R}$, $\mathcal{VO}$
// Graph Search
Initialize $\mathcal{P}$ with entry node v_{ep} keyed by $(\kappa_{\mathbf{q}}(v_{ep}), id_{v_{ep}})$;
$\mathcal{X} \leftarrow \emptyset$, $C \leftarrow \{v_{ep}\}$;
while $\mathcal{P} \neq \emptyset$ *and* $|\mathcal{X}| < E$ **do**
  $v \leftarrow \mathcal{P}.\text{PopMin}()$;
  if $v \notin \mathcal{X}$ **then**
    $\mathcal{X} \leftarrow \mathcal{X} \cup \{v\}$;
    foreach $u \in \mathcal{N}(v)$ **do**
      if $u \notin C$ **then**
        $C \leftarrow C \cup \{u\}$;
        $\mathcal{P}.\text{Push}(u, (\kappa_{\mathbf{q}}(u), id_u))$;
Compute $d(\mathbf{q}, \mathbf{x}_v)$ for all $v \in C$;
$\mathcal{R} \leftarrow \text{TopK}_{(d,id)}(C, k)$ and $\tau \leftarrow d(\mathbf{q}, \mathbf{x}_{r_k})$;
// Verification Information Generation
$\mathcal{F} \leftarrow \mathcal{R} \times \{1, \ldots, L\}$;
foreach $v \in C \setminus \mathcal{R}$ **do**
  $i \leftarrow 1$;
  while $i \le L$ *and* $LB_{b_i}(\mathbf{q}, v) \le \tau$ **do**
    $\mathcal{F} \leftarrow \mathcal{F} \cup \{(v, i)\}$;
    $i \leftarrow i + 1$;
$VO_{\text{trav}} \leftarrow \langle \{\mathsf{ER}(v)\}_{v \in \mathcal{X}}, \{\Delta_v^i\}_{(v,i) \in \mathcal{F}} \rangle$;
$\mathcal{F}_i \leftarrow \{v \mid (v, i) \in \mathcal{F}\}$, $1 \le i \le L$;
$\pi_{\text{route}} \leftarrow \text{MProof}(\mathcal{T}_{\text{route}}, \mathcal{X})$;
$\pi^i_{\text{ref}} \leftarrow \text{MProof}(\mathcal{T}^i_{\text{ref}}, \mathcal{F}_i)$, $1 \le i \le L$;
$VO_{\text{auth}} \leftarrow \langle \pi_{\text{route}}, \{\pi^i_{\text{ref}}\}_{i=1}^L \rangle$;
return $\mathcal{R}, \mathcal{VO} \leftarrow \langle VO_{\text{trav}}, VO_{\text{auth}} \rangle$;

Index Maintenance. QVANN operates on authenticated index snapshots. Upon dataset updates, the DO can rebuild the quantized graph and authentication trees and publish a new trusted digest. Each query is verified against a consistent snapshot. Efficient incremental maintenance is left for future work.

5.2 Verifiable Query Processing

Given a query $Q = \langle \mathbf{q}, k, E \rangle$, QVANN first performs graph traversal entirely at the coarsest precision and then generates the refinement information required for result certification. Alg. 1 gives the complete procedure.

Graph Search. The graph search contains three steps. (i) *Initialization.* The server initializes the priority queue $\mathcal{P}$ with the entry node specified by $\mathsf{M}_{\text{entry}}$, the expanded-node set $\mathcal{X}$, and the discovered candidate set $\mathcal{C}$ (lines 1–2). (ii) *Quantized graph traversal.* For each discovered node v, the routing score $\kappa_{\mathbf{q}}(v)$ is computed from its coarsest code $\mathbf{z}_v^{b_1}$ as defined in Sec. 4.2. The server repeatedly expands the top-1 unexpanded node ordered by $(\kappa_{\mathbf{q}}(v), id_v)$ and inserts newly discovered neighbors into $\mathcal{P}$ until E nodes are expanded or $\mathcal{P}$ becomes empty (lines 3–10). (iii) *Result generation.* The server computes the exact distances of all candidates in $\mathcal{C}$ and obtains the top-k result $\mathcal{R}$, where ties are broken by node ID and $\tau = d(\mathbf{q}, \mathbf{x}_{r_k})$ denotes the final result threshold (lines 11–12).

VO Generation. The verification information is generated after the candidate set is fixed. (i) *Progressive refinement.* All refinement levels of the returned results are included in $\mathcal{F}$ for exact reconstruction. For each $v \in \mathcal{C} \setminus \mathcal{R}$, the server progressively includes Δ_v^i while $LB_{b_i}(\mathbf{q}, v) \le \tau$. Once the lower bound exceeds τ, no finer representation is required. Otherwise, refinement continues until the exact distance is recovered, with ties resolved by node ID (lines 13–18). (ii) *VO generation.* The expansion records of $\mathcal{X}$ and the selected refinement records in $\mathcal{F}$ form VO_{trav}. The server then generates the routing multi-proof π_{route} over $\mathcal{X}$ and the refinement multi-proofs $\{\pi_{\text{ref}}^i\}_{i=1}^L$ over the corresponding sets $\{\mathcal{F}_i\}_{i=1}^L$, forming VO_{auth} (lines 19–23).

Example 3. *Fig. 5(a) illustrates an example query execution. (i) For the graph index, the expansion records of traversed nodes (green) and the required refinement information (orange) are included in VO_{trav}. (ii) For the authentication index, the corresponding Merkle proofs are included in VO_{auth}. For example, expanding v_7 requires authenticating $\mathsf{ER}(v_7)$ in $\mathcal{T}_{\text{route}}$. Since v_7 can only be safely excluded after its first refinement, $\Delta_{v_7}^1$ is also authenticated in $\mathcal{T}_{\text{ref}}^1$. These correspond to the green and orange highlighted entries, respectively.*

5.3 Verification

Given the trusted digest $\text{Digest} = \langle \mathsf{M}_{\text{entry}}, \mathsf{M}_{\text{quant}}, r_{\text{route}}, \{r_{\text{ref}}^i\}_{i=1}^L \rangle$ from the DO, the query results $\mathcal{R}$ and $\mathcal{VO} = \langle VO_{\text{trav}}, VO_{\text{auth}} \rangle$ from the SP, QU verifies the query result from two aspects: (i) correctness and (ii) completeness. Alg. 2 gives the complete procedure.

Correctness Verification. The correctness verification contains two steps. (i) *Authentication.* QU first extracts the trusted entry metadata and Merkle roots from Digest. It then parses the returned VOs and verifies π_{route} against r_{route} and each π_{ref}^i against r_{ref}^i, thereby authenticating all returned expansion and refinement records. The authenticated coarse codes are collected from the expansion records and entry metadata, with inconsistent copies rejected (lines 1–8). (ii) *Result verification.* QU first checks the result size and node IDs. For each $v \in \mathcal{R}$, QU checks that its authenticated coarse code and all required refinement levels are present, reconstructs the exact vector $\mathbf{x}_v$, and recomputes its distance to $\mathbf{q}$. The results are then ordered by $(d(\mathbf{q}, \mathbf{x}_v), id_v)$ to recover the exact threshold τ (lines 9–16).

Completeness Verification. The completeness verification also contains two steps. (i) *Traversal replay.* QU replays the quantized graph traversal using the trusted entry metadata, authenticated expansion records, and the same routing rule as Alg. 1. The replay reconstructs the candidate set $\mathcal{C}$ and rejects if any required expansion record is missing or inconsistent, or if $\mathcal{R} \not\subseteq \mathcal{C}$ (lines 17–19). (ii) *Candidate exclusion.* For each $v \in \mathcal{C} \setminus \mathcal{R}$, QU progressively checks $LB_{b_i}(\mathbf{q}, v)$ according to Lemma 1. If the current lower bound is insufficient for exclusion, QU uses the authenticated Δ_v^i to reconstruct the next precision level. If all quantized levels remain inconclusive, QU reconstructs the exact vector and verifies that v does not rank before the k-th result under the same distance-and-ID ordering (lines 20–31).

Algorithm 2: QVANN.Verify($Q, \mathcal{R}, \mathcal{VO}$, Digest)

Input: Query $Q = \langle \mathbf{q}, k, E \rangle$, result $\mathcal{R}$, $\mathcal{VO} = \langle VO_{\text{trav}}, VO_{\text{auth}} \rangle$, and trusted Digest $= \langle \mathsf{M}_{\text{entry}}, \mathsf{M}_{\text{quant}}, r_{\text{route}}, \{r_{\text{ref}}^i\}_{i=1}^L \rangle$
Output: ACCEPT or REJECT

`// Correctness Verification`

1 Parse $VO_{\text{trav}} = \langle \{\mathsf{ER}(v)\}, \{\Delta_v^i\}_{(v,i)\in\mathcal{F}} \rangle$ and $VO_{\text{auth}} = \langle \pi_{\text{route}}, \{\pi_{\text{ref}}^i\}_{i=1}^L \rangle$;
2 $\mathcal{F}_i \leftarrow \{v \mid (v,i) \in \mathcal{F}\}$, $1 \le i \le L$;
3 Verify π_{route} against r_{route} and π_{ref}^i against r_{ref}^i, $1 \le i \le L$;
4 **if** *any Merkle verification fails* **then**
5 **return** *REJECT*;
6 $\mathcal{Z} \leftarrow \text{CollectCodes}(\mathsf{M}_{\text{entry}}, \{\mathsf{ER}(v)\})$;
7 **if** *authenticated coarse codes are inconsistent* **then**
8 **return** *REJECT*;
9 **if** $|\mathcal{R}| \neq k$ *or* $\mathcal{R}$ *contains duplicate IDs* **then**
10 **return** *REJECT*;
11 **foreach** $v \in \mathcal{R}$ **do**
12 **if** $(v,i) \notin \mathcal{F}$ *for some* $1 \le i \le L$, *or* $v \notin \mathcal{Z}$ **then**
13 **return** *REJECT*;
14 Reconstruct $\mathbf{x}_v$ from $\mathcal{Z}[v]$ and $\{\Delta_v^i\}_{i=1}^L$;
15 Compute $d(\mathbf{q}, \mathbf{x}_v)$;
16 Sort $\mathcal{R}$ by $(d(\mathbf{q}, \mathbf{x}_v), id_v)$ and set $\tau \leftarrow d(\mathbf{q}, \mathbf{x}_{r_k})$;

`// Completeness Verification`

17 $\mathcal{C} \leftarrow \text{Replay}(\mathbf{q}, E, \mathsf{M}_{\text{entry}}, \{\mathsf{ER}(v)\})$;
18 **if** *replay fails or* $\mathcal{R} \not\subseteq \mathcal{C}$ **then**
19 **return** *REJECT*;
20 **foreach** $v \in \mathcal{C} \setminus \mathcal{R}$ **do**
21 $i \leftarrow 1$;
22 **while** $i \le L$ *and* $LB_{b_i}(\mathbf{q}, v) \le \tau$ **do**
23 **if** $(v,i) \notin \mathcal{F}$ **then**
24 **return** *REJECT*;
25 **if** $i < L$ **then**
26 $\mathbf{z}_v^{b_{i+1}} \leftarrow \text{Refine}(\mathbf{z}_v^{b_i}, \Delta_v^i)$;
27 **else**
28 $\mathbf{x}_v \leftarrow \text{Refine}(\mathbf{z}_v^{b_L}, \Delta_v^L)$;
29 **if** $(d(\mathbf{q}, \mathbf{x}_v), id_v) < (\tau, id_{r_k})$ **then**
30 **return** *REJECT*;
31 $i \leftarrow i + 1$;
32 **return** *ACCEPT*;

Example 4. *The verification procedure is shown in Fig. 5(b). (i) For correctness verification, the client authenticates the records in VO_{trav} using the proofs in VO_{auth}. For example, $\mathsf{ER}(v_2)$, $\Delta_{v_2}^1$, and $\Delta_{v_2}^2$ are authenticated against the corresponding Merkle roots r_{route}, r_{ref}^1, and r_{ref}^2. The returned results are then reconstructed to obtain the threshold τ from the k-th result $\mathbf{x}_{v_2}$. (ii) For completeness verification, the client replays the graph traversal using the authenticated coarse expansions (green nodes). For each non-result candidate, progressive exclusion is performed. Candidates such as v_1 and v_4 are directly excluded when $LB_{b_1}(\mathbf{q}, v) > \tau$, while candidates such as v_3 are refined using Δ_v^1 until exclusion is verified.*

6 ANALYSIS

In this section, we analyze the complexity and security of QVANN.

6.1 Complexity Analysis

We analyze the computational, communication, and storage costs of QVANN. Let n and d denote the number and dimension of vectors, respectively. For a query, let $\mathcal{X}$ and C be the expanded and discovered sets, with $x = |\mathcal{X}| \le E$ and $c = |C|$. Let $a = \sum_{v \in \mathcal{X}} |\mathcal{N}(v)|$ be the number of accessed neighbor entries, where $c \le a + 1$. For refinement, let $f_i = |\mathcal{F}_i|$, $f = \sum_{i=1}^{L} f_i$, and ϕ be the number of distance-bound evaluations. Let ι, s_0, s_i, and λ denote the bit sizes of a node ID, a coarse code, a level-i refinement payload, and a Merkle hash, respectively, where $\iota = \lceil \log_2(n+1) \rceil$.

Query Complexity. Graph traversal examines a neighbor entries and computes coarse distances for c candidates. Together with exact candidate distances and priority-queue operations, graph search takes $O(a+cd+c\log c)$ time. Progressive refinement requires $O(\phi d)$ time. With precomputed Merkle hashes, generating multi-proofs for $x+f$ records takes $O((x+f)\log n)$ time. Thus, the overall query complexity is $O(a + cd + c\log c + \phi d + (x+f)\log n)$.

Verification Complexity. QU authenticates $x + f$ records in $O((x+f)\log n)$ hash operations and replays graph traversal in $O(a + cd + c\log c)$ time. Progressive exclusion requires $O(\phi d)$ time, while reconstructing the k results costs $O(kLd)$. Thus, the overall verification complexity is $O(a+cd+c\log c+\phi d+kLd+(x+f)\log n)$.

VO Size. Each expanded node contributes one expansion record containing its ID and the IDs and coarse codes of its neighbors. Together with the disclosed refinement records, the traversal VO requires $O(x\iota + a(\iota + s_0) + \sum_{i=1}^{L} f_i(\iota + s_i))$ bits. For authentication, QVANN generates one routing multi-proof over x expansion records and up to L refinement multi-proofs over f_i records. The sibling-hash payload is bounded by $O((x+f)\lambda\log n)$ bits. Accounting for shared authentication paths, a tighter upper bound for balanced Merkle trees is $O(\lambda \sum_{i:t_i>0} t_i \log(1 + n/t_i))$, where $t_0 = x$ and $t_i = f_i$ for $1 \le i \le L$. These bounds exclude proof-position metadata, while the trusted digest is maintained separately. Notably, routing authentication requires only x proof targets rather than individual neighbor records.

Index Storage. Let $\bar{m}_{\mathcal{G}}$ denote the average graph degree. Adjacency lists require $O(n\bar{m}_{\mathcal{G}}\iota)$ bits, while coarse codes and refinement payloads occupy $O(n(s_0 + \sum_{i=1}^{L} s_i))$ bits. The $(L+1)$ Merkle trees require $O((L+1)n\lambda)$ bits. Accounting for node identifiers in the graph and refinement records, the total storage is $O(n[(\bar{m}_{\mathcal{G}} + L + 1)\iota + s_0 + \sum_{i=1}^{L} s_i + (L+1)\lambda])$. This bound assumes that expansion records are constructed from stored graph data on demand. Materializing independent expansion records would incur an additional $O(n\bar{m}_{\mathcal{G}}(\iota + s_0))$ bits.

6.2 Security Analysis

Following the threat model in Sec. 2, we now analyze the correctness and completeness of the verification procedure in QVANN. For a query $Q = \langle \mathbf{q}, k, E \rangle$, let C^* denote the candidate set generated by an honest execution of the prescribed quantized graph traversal. Let $\mathcal{R}^* = \textsc{TopK}_{(d,id)}(C^*, k)$ denote the correct top-k result set under the distance-and-ID ordering over C^*.

Definition 6 (Query Verification Correctness and Completeness). The verification procedure of QVANN is correct and complete if, for any probabilistic polynomial-time adversary $\mathcal{A}$, the probability that $\mathcal{A}$ succeeds in the following experiment is negligible:

- $\mathcal{A}$ selects a dataset $\mathcal{D}$ and a query $Q = \langle \mathbf{q}, k, E \rangle$.
- The DO constructs a well-formed authenticated QVANN index over $\mathcal{D}$ and provides the trusted Digest to QU. The adversary controls the CS.
- $\mathcal{A}$ outputs a result set $\mathcal{R}$ and verification object $\mathcal{VO}$.
- $\mathcal{A}$ succeeds if QVANN.Verify$(Q, \mathcal{R}, \mathcal{VO}, \text{Digest})$ returns Accept, while $\mathcal{R} \setminus \mathcal{R}^* \neq \emptyset$ (incorrect result) or $\mathcal{R}^* \setminus \mathcal{R} \neq \emptyset$ (incomplete result).

Theorem 2 (Correctness and Completeness of QVANN)**.** *Assuming that $H(\cdot)$ is collision-resistant and the DO provides a trusted digest of a well-formed authenticated index, results accepted by QVANN.Verify are both correct and complete, except with negligible probability.*

Proof. We show that any successful attack either violates the collision resistance of $H(\cdot)$ or contradicts the verification procedure. **Correctness:** $\{v \mid v \notin \mathcal{R}^* \wedge v \in \mathcal{R}\} \neq \emptyset$. An incorrect result can be introduced only through the following cases. (i) *Record forgery.* Modifying or replacing an authenticated expansion record $\mathsf{ER}(v)$ or refinement record Δ_v^i while preserving its Merkle root requires a hash collision. Thus, forged records are rejected except with negligible probability. (ii) *Result manipulation.* With authentic records, QU reconstructs the returned vectors and their exact distances, and verifies $\mathcal{R} \subseteq C$ and $|\mathcal{R}| = k$. Any incorrect inclusion therefore implies the omission of a required result, which is rejected by the completeness argument below.

Completeness: $\{v \mid v \in \mathcal{R}^* \wedge v \notin \mathcal{R}\} \neq \emptyset$. A required result can be omitted only through the following cases. (i) *Traversal manipulation.* QU replays traversal from the trusted entry using authenticated expansion records. Since each record contains complete neighbor information and routing is deterministic, QU reconstructs the honest candidate set $C = C^*$. Missing records cause replay to fail, while modified records violate authentication. (ii) *Refinement manipulation.* QU verifies every non-result candidate using authenticated quantization bounds and refinements. Missing required refinements cause rejection, while modified refinements violate authentication. (iii) *Result omission.* Suppose $v \in \mathcal{R}^* \setminus \mathcal{R}$. Since $|\mathcal{R}| = k$ and $\mathcal{R} \subseteq C^*$, we have $(d(\mathbf{q}, \mathbf{x}_v), id_v) < (\tau, id_{r_k})$. By Lemma 1, $LB_{b_i}(\mathbf{q}, v) \le d(\mathbf{q}, \mathbf{x}_v) \le \tau$ at every level. Thus, v cannot be excluded by quantized bounds and must undergo exact reconstruction, where its ranking causes verification to reject.

Thus, $\mathcal{R} = \mathcal{R}^*$ except with negligible probability, proving both correctness and completeness. □

7 EXPERIMENTS

7.1 Experiment Setup

Datasets. We evaluate four real-world vector datasets, summarized in Table 2. (i) SIFT: A collection of 128-dimensional local image descriptors from BIGANN/SIFT1B [4]. We use SIFT1M by default and prefixes of BIGANN/SIFT1B with up to 10 million vectors for scalability experiments. (ii) NYTimes: A dataset of 290,000 256-dimensional document vectors [1]. Removing 239 zero vectors leaves 289,761 objects. (iii) GloVe: A set of 1,183,514 100-dimensional

Table 2: Statistics of experimental datasets.

Dataset	Type (Dimension)	Original	Default	Max. Tested
SIFT	Image (128)	1,000,000,000	1,000,000	10,000,000
NYTimes	Document (256)	290,000	289,761	289,761
GloVe	Word (100)	1,183,514	1,183,514	1,183,514
DEEP	Deep image (96)	1,000,000,000	1,000,000	10,000,000

word embeddings [1, 26]. We use the full dataset. (iv) DEEP: A collection of 96-dimensional deep visual descriptors from DEEP1B [3]. We use DEEP1M by default and DEEP1B prefixes with up to 10 million vectors for scalability experiments. For NYTimes and GloVe, data and query vectors are normalized to unit length, and squared Euclidean distance preserves cosine-similarity rankings.

Algorithms. We compare QVANN against four representative baselines. (i) V3DB [30], a ZK-based method that encodes IVF-PQ search into optimized zero-knowledge circuits to verify ANN query execution[1]. (ii) MR-tree [8], a traditional low-dimensional data ADS-based method that authenticates exact kNN queries using a Merkle R-tree. We adapt its non-confidential variant to high-dimensional Euclidean vectors. (iii) VLAH [36], an ADS-based method that combines LSH bucketing, compressed-trie routing, local HNSW search, and Merkle authentication for verifiable ANN search. (iv) ANNProof [20], an ADS-based method that authenticates graph-based ANN search using Merkle proofs.

Setup. All indexes are implemented in C++, and experiments are conducted on a Linux workstation equipped with an Intel Xeon Platinum 8369C CPU (2.90 GHz) and 512 GB RAM. Merkle authentication uses SHA-256. ANNProof and QVANN share the same NSW graph. By default, the ANN query result size $k = 10$, and QVANN employs SQ4-based top-1 traversal with progressive SQ4–SQ6–raw refinement. Search budgets are tuned to achieve approximately 95% Recall@10, with the actual recall reported. Query processing and verification are single-threaded. We report Recall@k (R), query time (Q), VO size (VO), and verification time (V). Query time includes search and VO generation, while verification time includes VO decoding, authentication, traversal replay, and result checking.

7.2 Overall Performance

We use the default index settings and tune the search parameters of ADS-based ANN methods to achieve comparable recall (around 95%). For ANNProof and QVANN, we set $(M, efConstruction) = (8, 100)$ on SIFT and $(16, 200)$ on the other datasets. The Merkle shard size of ANNProof and the LSH bucket capacity of VLAH are set to 10,000 and 20,000, respectively. Table 3 reports the overall performance across four datasets. We make the following observations. **(i) Recall.** The ADS-based ANN methods, including ANNProof, VLAH, and QVANN, achieve at least 95% Recall@10 across all datasets. In contrast, V3DB achieves only 76.0% and 71.5% on NYTimes and GloVe, respectively. Higher recall requires probing more inverted lists, substantially increasing circuit complexity and prover memory consumption. Even at 76.0% recall on NYTimes, V3DB query evaluation reports 447.2 GB of peak prover memory under a separate large-circuit configuration [30]. Meanwhile, MR-tree achieves 100% recall through authenticated exact kNN search. **(ii) VO size.** QVANN produces the smallest VOs among ADS-based methods, even outperforming the ZK-based V3DB on SIFT and DEEP by 58.2% and 60.3%, respectively. Compared with the state-of-the-art ADS-based ANNProof, QVANN reduces VO size by 88.3%–90.7% across four datasets. The largest reduction occurs on NYTimes, where VO size drops from 13,666 KB to 1,271 KB. **(iii) Verification time.** Benefiting from compact authentication proofs, QVANN achieves the fastest verification among ADS-based methods, outperforming ANNProof by 2.9–4.2×, VLAH by 9.3–45.9×, and MR-tree by 3.5–65.3×. Compared with V3DB, it achieves 1.6–13.0× faster verification on three datasets, while V3DB is faster on GloVe, but at substantially lower recall. **(iv) Query time.** QVANN consistently achieves the lowest query time among ADS-based ANN methods, outperforming VLAH by 8.1–42.6×. In contrast, V3DB requires 2,027.7–4,949.5 s per query due to expensive ZK proof generation.

7.3 Recall–Cost Trade-off

We examine the recall–cost trade-off by varying (i) $efSearch$ in ANNProof from 14 to 512 on SIFT, 10 to 96 on DEEP, and 64 to 8192 on GloVe; (ii) the expansion budget E in QVANN over 22–512, 16–128, and 64–8192; (iii) the number of probed LSH buckets in VLAH over 11–47, 9–47, and 26–60, respectively; and (iv) the IVF probing budget in V3DB. MR-tree is excluded due to its exact kNN search. Figs. 6–8 report the results. **(i) VO size.** QVANN consistently achieves substantially lower communication costs across different recall levels. At around 95% recall, it generates up to 141.4× smaller VOs than VLAH. This advantage remains significant at higher recall. At around 99% recall, QVANN reduces VO size by 43.8× and 100.8× compared with VLAH on SIFT and DEEP, respectively. Even against ANNProof, it produces a 6.3× smaller VO on SIFT. These improvements benefit from coarse-precision candidate exclusion and selective vector refinement. **(ii) Verification time.** QVANN also maintains efficient verification at high recall. At around 99% recall, it achieves 14.4× and 31.7× faster verification than VLAH on SIFT and DEEP, respectively. This advantage stems from reduced vector disclosure and compact Merkle authentication proofs. **(iii) Query time.** QVANN maintains low query costs as recall increases. At around 99% recall, it outperforms VLAH by 10.4× on SIFT and 28.4× on DEEP. Coarse-precision traversal and reduced VO generation overhead enable efficient high-recall queries.

7.4 Scalability

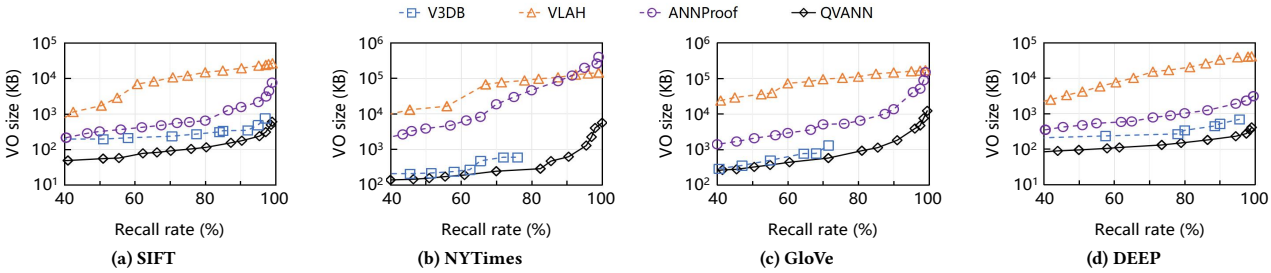
We evaluate scalability on SIFT and DEEP with 0.1–10 million vectors. Figs. 9–11 present the results. (i) **VO size.** At 10 million vectors, QVANN generates only 723.1 KB and 651.6 KB on SIFT and DEEP, reducing VO size by 85.3% and 87.8% over ANNProof, respectively. In contrast, MR-tree incurs over 3 GB of VOs on both datasets. (ii) **Verification time.** QVANN maintains the lowest verification cost among ADS-based methods, requiring only 17.2 ms and 15.8 ms at 10 million vectors. (iii) **Query time.** QVANN consistently achieves low query latency, remaining below 11 ms even at the largest scale. For V3DB, both measured and estimated results are limited to 1 million vectors, as its proof generation already consumes 129.4 GB of memory on SIFT at this scale.

[1]Our implementation is adapted from the authors' open-source code: https://github.com/Relaxed-System-Lab/V3DB.

Table 3: Overall performance comparison across four datasets.

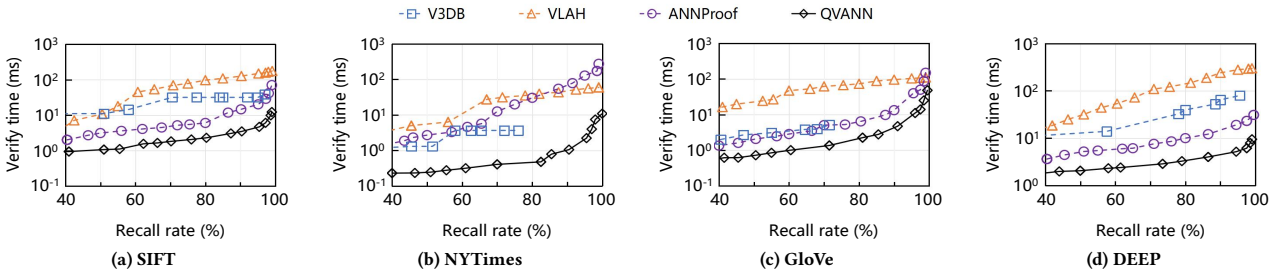
Method	SIFT				NYTimes				GloVe				DEEP			
	R (%)	Q (ms)	VO (KB)	V (ms)	R (%)	Q (ms)	VO (KB)	V (ms)	R (%)	Q (ms)	VO (KB)	V (ms)	R (%)	Q (ms)	VO (KB)	V (ms)
ANNProof	95.1	2.8	1,885	13.5	95.0	17.7	13,666	92.8	95.5	83.7	40,574	395.9	97.5	3.8	2,310	23.1
VLAH	96.3	36.9	13,951	101.3	96.0	295.9	135,489	556.6	95.0	420.8	156,085	1,029	95.0	106.5	38,322	280.1
V3DB	95.0	2,405,603	483.4†	31.9	76.0	2,027,663	597.2†	36.4	71.5	4,949,536	1,294†	51.3	95.5	2,361,625	683.2†	79.5
MR-tree	100.0	278.3	235,214	202.1	100.0	204.6	282,959	243.2	100.0	468.7	466,933	390.3	100.0	423.3	373,923	398.2
QVANN	96.4	2.4	202.2	4.7	96.5	10.8	1,271	22.2	96.0	52.1	3,832	110.4	97.5	2.5	271.1	6.1

Notes. R: Recall@10; Q: query time; VO: verification object size; V: verification time. These abbreviations are used throughout the experiments. † On NYTimes and GloVe, V3DB produces smaller VOs but achieves substantially lower recall (76.0% and 71.5%, respectively), making these results not directly comparable.



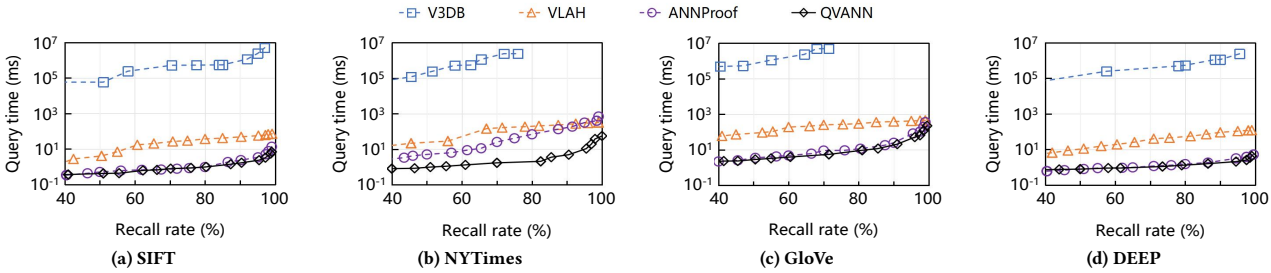
(a) SIFT (b) NYTimes (c) GloVe (d) DEEP

Figure 6: VO size under varying Recall@10.



(a) SIFT (b) NYTimes (c) GloVe (d) DEEP

Figure 7: Verification time under varying Recall@10.



(a) SIFT (b) NYTimes (c) GloVe (d) DEEP

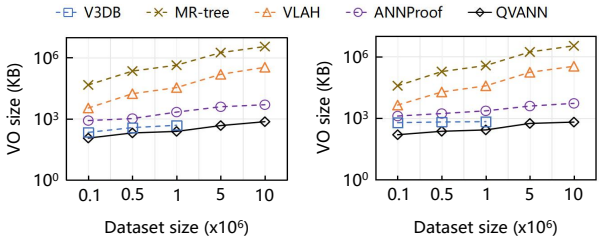
Figure 8: Query time under varying Recall@10.

7.5 Impact of Result Size

We vary the result size k from 1 to 100 on SIFT and DEEP, with search parameters fixed at settings tuned for approximately 95% Recall@10. Figs. 12–14 present the results. (i) **VO size.** Despite increasing k, QVANN maintains VO sizes below 500 KB on both datasets. At $k = 100$, it reduces VO size by 79.6% and 85.3% over ANNProof, and by 98.1% and 98.7% over VLAH, respectively. (ii) **Verification time.** At $k = 100$, QVANN requires only 9.1 ms and 10.3 ms on SIFT and DEEP, outperforming VLAH by 18.0× and 27.3×, respectively. (iii) **Query time.** QVANN maintains query latency below 4 ms even at $k = 100$, demonstrating efficient query processing as the result size increases.

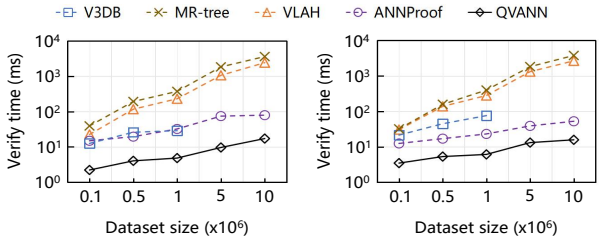
7.6 Index Size and Construction Time

Fig. 15 compares index storage and construction costs across all datasets. (i) **Index size.** QVANN requires 497.1–1,145.0 MB of storage, 19.4%–29.0% more than ANNProof, due to additional multi-level quantization data and separately authenticated routing and refinement records. (ii) **Construction time.** QVANN takes 252.8–859.9 s



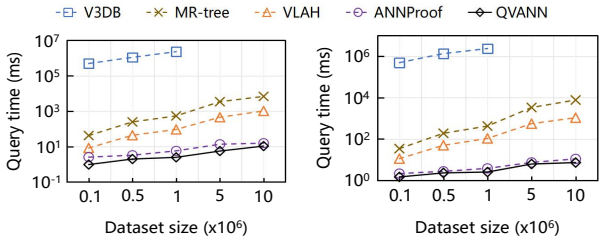
(a) SIFT (b) DEEP

Figure 9: VO size under varying dataset size.



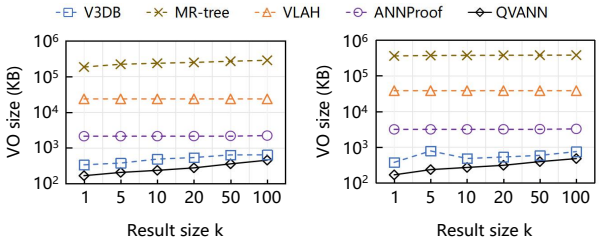
(a) SIFT (b) DEEP

Figure 10: Verification time under varying dataset size.



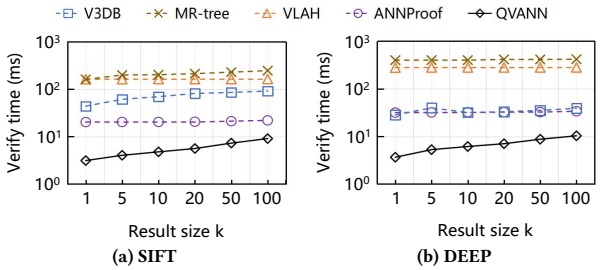
(a) SIFT (b) DEEP

Figure 11: Query time under varying dataset size.



(a) SIFT (b) DEEP

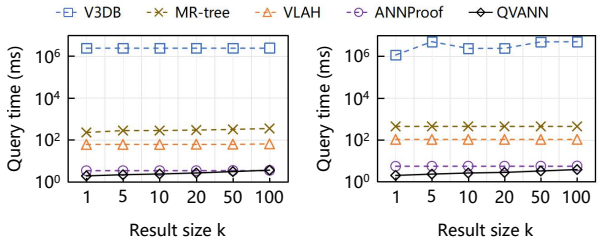
Figure 12: VO size under varying result size k.



(a) SIFT (b) DEEP

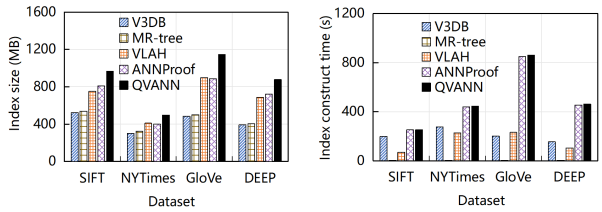
Figure 13: Verification time under varying result size k.

for index construction, incurring only 0.3%–1.6% overhead over ANNProof. These moderate offline costs are justified by the substantial reductions in online VO size and verification time.



(a) SIFT (b) DEEP

Figure 14: Query time under varying result size k.



(a) Index size (b) Construction time

Figure 15: Index storage and construction costs.

7.7 Ablation Study

Table 4 evaluates the contributions of the three components of QVANN. To assess the generality of progressive quantization, we additionally construct IVF-ADS and NSW-ADS by combining conventional ANN indexes with Merkle authentication, along with their quantized variants incorporating (i). (i) **Progressive quantization.** Applying (i) to IVF-ADS reduces VO size by 58.1%–79.7% across four datasets, demonstrating its applicability beyond graph-based indexes. In contrast, applying (i) alone to NSW-ADS provides limited VO savings and increases verification time on all datasets, as exact-distance traversal requires repeated vector refinement. (ii) **Verification-decoupled traversal.** Adding (ii) to quantized NSW reduces VO size by 59.4%–76.0% and verification time by 39.3%–64.4%. By eliminating fine-grained refinement during traversal, it enables graph-based search to better exploit progressive quantization. (iii) **Expansion-centric authentication.** Adding (iii) further reduces VO size by 49.2%–61.2% and verification time by 53.2%–60.0%. These improvements result from expansion-level routing authentication and shared Merkle proof paths. Overall, QVANN reduces VO size by 82.1%–91.9% over NSW-ADS.

7.8 Impact of Quantization Levels

We evaluate five quantization configurations across four datasets, including Raw and four quantized variants. The quantized variants share SQ4-based top-1 traversal but differ in refinement precision and depth. Table 5 reports the results. Compared with Raw, quantized variants reduce VO size by 37.8%–82.4%. Introducing an intermediate SQ6 level further reduces VO size by 25.9%–58.9% over SQ4–raw, highlighting the benefits of progressive refinement. Although multi-level refinement introduces additional verification costs, the differences among multi-level configurations remain relatively small. In particular, the default SQ4–SQ6–raw achieves VO sizes within 4.6% of the minimum across all datasets, with verification times at most 0.3 ms above the fastest quantized configuration.

Table 4: Ablation study across four datasets. (i) Progressive quantization; (ii) verification-decoupled traversal; (iii) expansion-centric authentication. QVANN incorporates all three components.

Variant	SIFT				NYTimes				GloVe				DEEP			
	R (%)	Q (ms)	VO (KB)	V (ms)	R (%)	Q (ms)	VO (KB)	V (ms)	R (%)	Q (ms)	VO (KB)	V (ms)	R (%)	Q (ms)	VO (KB)	V (ms)
IVF-ADS	95.0	128.9	24,692	266.4	96.0	897.5	171,156	913.8	96.0	656.1	114,537	1,451	95.0	102.8	19,218	260.5
IVF-ADS+(i)	95.0	47.6	8,019	241.2	96.0	223.0	34,822	909.6	96.0	285.0	33,296	1,447	95.0	51.4	8,058	259.7
NSW-ADS	95.0	2.9	968.2	6.4	96.0	44.3	15,696	70.4	95.0	71.7	21,571	173.6	95.0	4.3	1,310	10.1
NSW-ADS+(i)	95.0	2.6	713.8	10.0	96.0	39.6	10,447	130.9	95.0	81.7	20,580	340.8	95.0	5.1	1,361	19.1
(i)+(ii)	95.5	2.0	289.9	5.5	95.5	17.2	2,504	46.6	96.0	58.4	7,204	180.9	95.0	4.2	531.3	11.6
QVANN	95.5	1.1	112.4	2.2	95.5	10.9	1,271	21.8	96.0	35.9	2,804	77.8	95.0	2.3	234.5	5.4

Table 5: Impact of quantization configurations.

Configuration	SIFT		NYTimes		GloVe		DEEP	
	VO (KB)	V (ms)	VO (KB)	V (ms)	VO (KB)	V (ms)	VO (KB)	V (ms)
Raw	906.8	4.1	6,983	20.8	16,839	101.8	718.9	3.2
SQ4–raw	273.0	4.4	3,093	35.8	7,291	139.3	447.0	7.1
SQ4–SQ6–raw	202.2	4.0	1,271	21.5	3,832	109.9	271.1	6.0
SQ4–SQ8–raw	198.0	3.9	1,331	21.2	3,995	109.7	259.4	5.7
SQ4–SQ6–SQ8–raw	201.5	4.2	1,229	21.5	3,804	110.6	264.4	6.2

Table 6: Impact of routing precision.

Setting	Routing	SIFT				DEEP			
		E	R (%)	VO (KB)	V (ms)	E	R (%)	VO (KB)	V (ms)
Fixed E	Exact	128	98.0	968.4	13.1	64	96.0	941.3	15.9
	SQ4	128	98.0	202.2	4.4	64	97.5	271.1	6.7
Matched recall R	Exact	88	95.0	716.7	9.8	56	95.0	849.5	14.3
	SQ4	111	95.0	185.0	4.1	49	95.0	231.4	5.7

7.9 Impact of Routing Precision

We evaluate exact-distance and SQ4-based routing under fixed expansion budgets and matched recall, as shown in Table 6. Under fixed E, SQ4 routing reduces VO size by 79.1% and 71.2%, and verification time by 66.4% and 57.9% on SIFT and DEEP, respectively, while maintaining or improving recall. At matched 95.0% recall, it achieves 74.2% and 72.8% smaller VOs and 58.2% and 60.1% lower verification time, respectively. These results demonstrate that coarse-precision routing substantially reduces verification overhead by eliminating exact-distance comparisons during traversal, without compromising retrieval accuracy.

8 RELATED WORK

8.1 Outsourced Vector Databases

ANN search underpins modern vector databases, whose widely deployed indexes mainly follow two designs. (i) *Graph-based indexes*, including NSW, HNSW, NSG, and DiskANN, organize vectors into proximity graphs and explore a small, query-dependent neighborhood [9, 21, 22, 34]. (ii) *IVF-based indexes* partition vectors into clusters and search only selected inverted lists [15, 16]. Both designs can incorporate quantization to compress vectors and accelerate approximate distance evaluation [10, 11, 15].

With the growing demand for large-scale semantic retrieval, managed vector database services, such as Pinecone, Zilliz Cloud, Weaviate Cloud, and Qdrant Cloud, enable users to outsource vector storage, index maintenance, and query processing [27, 28, 38, 46]. This deployment model raises important security concerns. Existing studies protect data and query privacy through secure computation and oblivious retrieval [7, 18, 32, 44, 45], while embedding-inversion attacks further highlight the sensitivity of exposed vectors [25]. Beyond confidentiality, the integrity of outsourced retrieval is critical: adversarial corpus injection can manipulate downstream answers, motivating defenses based on authenticated provenance [33, 47]. However, authenticating returned objects alone cannot establish that an untrusted provider faithfully executes the prescribed ANN algorithm without substituting or omitting results. This motivates *verifiable ANN query processing*, which enables clients to validate search results against a committed index.

8.2 Verifiable Query Processing

Verifiable query processing has been widely studied in outsourced databases and blockchain systems, mainly following two approaches. (i) *ADS-based methods* combine Merkle hashes and digital signatures with query indexes [23], supporting range, keyword, spatial kNN, and image retrieval queries [13, 40–42], as well as blockchain aggregates [35]. (ii) *Algebraic and proof-based methods* employ accumulators and vector or polynomial commitments [5, 6, 17]. Representative systems include vChain and vChain+ for Boolean range queries [37, 39], and vSQL, ZKSQL, and PoneglyphDB for general database computation [12, 19, 43]. These approaches offer different trade-offs among proof size, prover computation, and interaction.

Recent studies extend verification to ANN retrieval. (i) *ZK-based approaches*, including V3DB for IVF-PQ and Atlas for HNSW, prove search execution over committed indexes while hiding private index contents, but incur huge additional proof-generation costs [2, 30]. (ii) *ADS-based approaches* authenticate graph records or segments using Merkle structures [20, 29], or combine LSH bucketing with local HNSW search and hierarchical authentication [36]. Related designs authenticate retrieved-document provenance [33] or combine Merkle commitments with secure computation for private LSH-based retrieval [14]. Despite existing proof-compression techniques, graph-based verification still requires substantial vector information and authentication evidence to replay traversal. QVANN reduces these costs through progressive quantization, low-precision routing decoupled from exact candidate ranking, and expansion-centric authentication.

9 CONCLUSION

We presented QVANN, a quantization-driven authenticated index for efficient verifiable ANN search over outsourced vector databases. QVANN combines progressive vector quantization, verification-decoupled graph traversal, and expansion-centric authentication to reduce both vector disclosure and authentication overhead. Extensive experiments demonstrate that QVANN outperforms state-of-the-art methods with up to 90.7% smaller VOs and 4.2× faster verification at comparable recall.

REFERENCES

[1] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2020. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. *Information Systems* 87 (2020), 101374.

[2] Nikolay Avramov, Hidde Lycklama, Alexander Viand, and Anwar Hithnawi. 2026. Atlas: Efficient Verifiable Semantic Search. *arXiv Preprint* (2026). https://doi.org/10.48550/arXiv.2609.11841

[3] Artem Babenko and Victor Lempitsky. 2016. Efficient Indexing of Billion-Scale Datasets of Deep Descriptors. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2055–2063.

[4] Big ANN Benchmarks. 2021. NeurIPS 2021 Billion-Scale Approximate Nearest Neighbor Search Challenge: Datasets. https://big-ann-benchmarks.com/neurips21.html

[5] Jan Camenisch and Anna Lysyanskaya. 2002. Dynamic Accumulators and Application to Efficient Revocation of Anonymous Credentials. In *Advances in Cryptology—CRYPTO 2002*. 61–76.

[6] Dario Catalano and Dario Fiore. 2013. Vector Commitments and Their Applications. In *Public-Key Cryptography 2013*.

[7] Hao Chen, Ilaria Chillotti, Yihe Dong, Oxana Poburinnaya, Ilya Razenshteyn, and M. Sadegh Riazi. 2020. SANNS: Scaling Up Secure Approximate k-Nearest Neighbors Search. In *Proceedings of the 29th USENIX Security Symposium*. 2111–2128.

[8] Qian Chen, Haibo Hu, and Jianliang Xu. 2013. Authenticating Top-k Queries in Location-based Services with Confidentiality. *Proceedings of the VLDB Endowment* 7 (2013), 49–60.

[9] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *Proceedings of the VLDB Endowment* 12 (2019), 461–474.

[10] Jianyang Gao and Cheng Long. 2024. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2 (2024).

[11] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2013. Optimized Product Quantization for Approximate Nearest Neighbor Search. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2946–2953.

[12] Binbin Gu, Juncheng Fang, and Faisal Nawab. 2025. PoneglyphDB: Efficient Non-interactive Zero-Knowledge Proofs for Arbitrary SQL-Query Verification. *Proceedings of the ACM on Management of Data* 3 (2025).

[13] Shangwei Guo, Jianliang Xu, Ce Zhang, Cheng Xu, and Tao Xiang. 2019. ImageProof: Enabling Authentication for Large-Scale Image Retrieval. In *Proceedings of the 35th IEEE International Conference on Data Engineering*. 1070–1081.

[14] Xiangyu Hui, Xingliang Yuan, Olga Ohrimenko, and Sid Chi-Kin Chau. 2026. Practical and Verifiable Encrypted Vector Search for Retrieval-Augmented Generation. *Cryptology ePrint Archive* 2026 (2026).

[15] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33 (2011), 117–128.

[16] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7 (2021), 535–547.

[17] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. 2010. Constant-Size Commitments to Polynomials and Their Applications. In *Advances in Cryptology—ASIACRYPT 2010*. 177–194.

[18] Jingyu Li, Zhicong Huang, Min Zhang, Cheng Hong, Jian Liu, Tao Wei, and Wenguang Chen. 2025. Panther: Private Approximate Nearest Neighbor Search in the Single Server Setting. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*. 365–379.

[19] Xiling Li, Chenkai Weng, Yongxin Xu, Xiao Wang, and Jennie Rogers. 2023. ZKSQL: Verifiable and Efficient Query Evaluation with Zero-Knowledge Proofs. *Proceedings of the VLDB Endowment* 16 (2023), 1804–1816.

[20] Lingling Lu, Zhenyu Wen, Ye Yuan, Qinming He, Jianhai Chen, and Zhenguang Liu. 2024. ANNProof: Building a Verifiable and Efficient Outsourced Approximate Nearest Neighbor Search System on Blockchain. *Future Generation Computer Systems* 156 (2024), 206–220.

[21] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate Nearest Neighbor Algorithm Based on Navigable Small World Graphs. *Information Systems* 45 (2014), 61–68.

[22] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42 (2020), 824–836.

[23] Ralph C. Merkle. 1988. A Digital Signature Based on a Conventional Encryption Function. In *Advances in Cryptology—CRYPTO 1987*. 369–378.

[24] Jason Mohoney, Devesh Sarda, Mengze Tang, Shihabur Rahman Chowdhury, Anil Pacaci, Ihab F. Ilyas, Theodoros Rekatsinas, and Shivaram Venkataraman. 2025. Quake: Adaptive Indexing for Vector Search. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation*. 153–169.

[25] John Xavier Morris, Volodymyr Kuleshov, Vitaly Shmatikov, and Alexander M. Rush. 2023. Text Embeddings Reveal (Almost) As Much As Text. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 12448–12460.

[26] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. GloVe: Global Vectors for Word Representation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 1532–1543.

[27] Pinecone. 2026. Pinecone: Fully Managed Vector Database. https://www.pinecone.io/

[28] Qdrant. 2026. Qdrant Managed Cloud. https://qdrant.tech/documentation/cloud/

[29] Chaoxia Qin, Yixiong Tang, Bing Guo, Kan Zhong, and Duo Liu. 2026. CD-ANN: Scalable Approximate Nearest Neighbor Search on Client-Side Devices. *Journal of Systems Architecture* 175 (2026), 103771.

[30] Zipeng Qiu, Wenjie Qu, Jiaheng Zhang, and Binhang Yuan. 2026. V3DB: Audit-on-Demand Zero-Knowledge Proofs for Verifiable Vector Search over Committed Snapshots. *arXiv Preprint* (2026). https://doi.org/10.48550/arXiv.2603.03065

[31] Derrick Quinn, Mohammad Nouri, Neel Patel, John Salihu, Alireza Salemi, Sukhan Lee, Hamed Zamani, and Mohammad Alian. 2025. Accelerating Retrieval-Augmented Generation. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Vol. 1. 15–32.

[32] Sacha Servan-Schreiber, Simon Langowski, and Srinivas Devadas. 2022. Private Approximate Nearest Neighbor Search with Sublinear Communication. In *Proceedings of the IEEE Symposium on Security and Privacy*. 911–929.

[33] Shivani Shukla and Himanshu Joshi. 2025. Proof-Carrying Answers: A Systematic Protocol for Verifiable Retrieval-Augmented Generation with Cryptographic Provenance. In *Annual Computer Security Applications Conference Workshops*. 405–413.

[34] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems*, Vol. 32.

[35] Weijie Sun, Zihuan Xu, Wangze Ni, Lei Chen, Peng Cheng, and Chen Jason Zhang. 2025. Authenticated Aggregate Queries with Boolean Range Predicates on Blockchains. *Proceedings of the VLDB Endowment* 18 (2025), 3615–3627.

[36] Yiping Teng, Yuyao Tang, Gang Wang, Shiqing Wang, Changze Li, and Liang Zhao. 2025. VLAH: A Lightweight and Verifiable Framework for Approximate Nearest Neighbor Search in High-Dimensional Space. In *IEEE 24th International Conference on Trust, Security and Privacy in Computing and Communications*. 1608–1616.

[37] Haixin Wang, Cheng Xu, Ce Zhang, Jianliang Xu, Zhe Peng, and Jian Pei. 2022. vChain+: Optimizing Verifiable Blockchain Boolean Range Queries. In *Proceedings of the 38th IEEE International Conference on Data Engineering*. 1928–1941.

[38] Weaviate. 2026. Weaviate Cloud. https://docs.weaviate.io/cloud

[39] Cheng Xu, Ce Zhang, and Jianliang Xu. 2019. vChain: Enabling Verifiable Boolean Range Queries over Blockchain Databases. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 141–158.

[40] Ce Zhang, Cheng Xu, Haixin Wang, Jianliang Xu, and Byron Choi. 2021. Authenticated Keyword Search in Scalable Hybrid-Storage Blockchains. In *Proceedings of the 37th IEEE International Conference on Data Engineering*. 996–1007.

[41] Ce Zhang, Cheng Xu, Jianliang Xu, and Byron Choi. 2018. Distributed kNN Query Authentication. In *Proceedings of the 19th IEEE International Conference on Mobile Data Management*. 167–176.

[42] Ce Zhang, Cheng Xu, Jianliang Xu, Yuzhe Tang, and Byron Choi. 2019. GEM^2-Tree: A Gas-Efficient Structure for Authenticated Range Queries in Blockchain. In *Proceedings of the 35th IEEE International Conference on Data Engineering*. 842–853.

[43] Yupeng Zhang, Daniel Genkin, Jonathan Katz, Dimitrios Papadopoulos, and Charalampos Papamanthou. 2017. vSQL: Verifying Arbitrary SQL Queries over Dynamic Outsourced Databases. In *Proceedings of the IEEE Symposium on Security and Privacy*. 863–880.

[44] Mingxun Zhou, Elaine Shi, and Giulia Fanti. 2025. Pacmann: Efficient Private Approximate Nearest Neighbor Search. In *Proceedings of the Thirteenth International Conference on Learning Representations*.

[45] Jinhao Zhu, Liana Patel, Matei Zaharia, and Raluca Ada Popa. 2025. Compass: Encrypted Semantic Search with High Accuracy. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation*. 915–938.
[46] Zilliz. 2026. Zilliz Cloud Developer Hub. https://docs.zilliz.com/docs/home
[47] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. 2025. PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models. In *Proceedings of the 34th USENIX Security Symposium*.